

METRON — LLM Token Usage Audit & Reduction Plan

Where every token is spent across the 8-stage pipeline, and how to cut cost & call volume drastically

1. Executive Summary

LLM cost is concentrated almost entirely in **Stage 4 (evaluation)**. The dominant driver is **DeepEval metric fan-out evaluated per conversation turn**, not RCA. Two structural facts make the problem worse than it looks:

- **RCA makes zero LLM calls** — `run_rca()` is fully deterministic taxonomy scoring. The token-heavy step people attribute to RCA is the per-prompt failure classifier in Stage 8.
- **The cost meter was blind.** Every DeepEval metric call bypassed the LLMClient accumulators, so the tracked `token_summary` under-reported the true bill by a wide margin. The biggest spender was invisible. (This audit ships with a fix that routes 100% of calls through one counter — see the LLMOps change log.)

With the tracking fix in place, the realistic reduction target is **60–80%** of evaluation tokens through consolidation + de-duplication, plus a further **30–50%** on what remains through tiered model routing and caching.

■ **Most important single finding:** DeepEval (hallucination, answer_relevancy, usefulness, bias, all GEval quality criteria) called the provider directly via its own Azure/litellm client at `max_tokens=2048` with its own retry loop — never counted, never rate-limited by the shared limiter. This is both the largest cost and the largest measurement gap.

2. Token Usage Map — every LLM call site

Multiplier is what matters: a call “per turn” nested inside “per conversation” explodes fastest. “Real calls” counts DeepEval internal fan-out (each metric = 2–4 provider calls).

Stage	Call site	Runs...	Real calls	Out cap
s0 profile	document_parser / technical_extractor	once/run — cached by doc SHA	0–2	—
s0 arch	architecture_parser (vision)	only /parse-architecture	1	2048
s1 personas	persona_builder (+1 revision, retries=3)	per persona	1–3	2000
s1	coverage_validator	once/run	1	—
s2 func	functional_gen	per persona — skipped if scenario exists	0–1	1200
s2 sec	security_gen_adapt_attack	per adv. persona × cat × ≤2	up to 12/persona	600
s2 qual	quality_criteria	once/run	1	—
s3 exec	conversation_runner combined turn	per dynamic turn	1/turn	600
s4 func	hallucination + answer_relevancy + judge	PER TURN	~6/turn	2048
s4 func	usefulness + completeness + cross_turn	per conversation	~4	2048
s4 qual	GEval	per criterion × conversation	~2/criterion	2048
s4 sec	bias	per non-security conversation	~2	2048
s4 sec	attack_resistance + toxic_request	per security conv (0 if refused)	0–1	500
s4 rag	relevancy+correctness+halluc+complete+factual	per RAG ground-truth conv	up to 5+	150–2048
s4 garak	resistance_verdict	per probe (~17)	1	uncapped
s8 classify	prompt_classifier	per batch of 5 failures	1	1500

Stage	Call site	Runs...	Real calls	Out cap
s8 RCA	run_rca	—	0 (deterministic)	—

Illustrative cost of ONE 3-turn functional conversation

Counting DeepEval fan-out at its 2048-token output ceiling:

Metric group	Where	Real calls
hallucination ×3 turns	functional.py	~6
answer_relevancy ×3 turns	functional.py	~9
llm_judge ×3 turns	functional.py (LLMClient)	3
usefulness + completeness + cross_turn	functional.py	~4
bias	security.py (passive)	~2
GEval × ~6 quality criteria	quality.py	~12
TOTAL for ONE conversation		~36 calls

~33 of those 36 were invisible to the old tracker. Multiply by (personas × prompts) conversations and Stage 4 is 80–90% of the bill.

3. Duplicated & Wasted Work (“same work again and again”)

- **/api/preview throws away a full persona set.** preview runs build_all_personas (N×2000-token calls) + scenario generation, then /api/run regenerates personas from scratch. Preview output is never reused → persona generation happens twice for any user who previews then runs.
- **functional.py and rag.py double-evaluate the same RAG conversations.** answer_relevancy, hallucination and completeness are each computed twice by two different toolchains on identical conversations.
- **Per-turn evaluation.** hallucination + relevancy + judge run on every turn though the meaningful signal is the final turn — a 2–3× multiplier on the most expensive stage.
- **Bias runs on every non-security conversation** — a full DeepEval metric on functional convs that already cost ~30 calls.
- **quality.py GEval re-scores functional conversations** once per criterion (6 criteria = ~12 calls/conv).
- **Persona revision + JSON retries.** persona_builder does generate (2000 tok) + conditional revision (2000 tok), and retries=3 on JSON-parse can triple a single persona.
- **Garak verdict has no max_tokens** — a one-word RESISTED/COMPLIED answer is billed at the full default budget.

4. Two Cost Amplifiers Baked Into the Client

(a) The model-tier selector is dead. Call sites pass task="fast"|"judge"|"balanced", but LLMClient ignores it (candidates = [self.model]). Cheap mechanical work (persona-turn simulation, attack adaptation, failure classification, garak verdict) runs on the same expensive judge model.

(b) No caching of any kind. No prompt cache, no dedup. The 133-point taxonomy text is re-sent on every classifier batch; identical static system prompts are re-sent on every eval call.

(c) Stage-4 thundering herd. functional + security + quality run concurrently, each spawning DeepEval threads with their own uncoordinated retry loop → provider 429s → retries → tokens paid for failed calls that produce nothing.

5. How to Reduce — ranked by impact

Tier 1 — Structural (target 60–80% of eval tokens)

#	Change	Effect
A	Collapse the per-conversation metric swarm into ONE judge call returning all sub-scores as JSON; route it through LLMClient (counted + rate-limited).	~36 → ~1–2 calls/conv. Biggest lever; also kills retry storms.
B	Evaluate the LAST turn only (or turn-1 for single-shot metrics) instead of per turn.	2–3× off Stage 4.
C	In RAG mode, skip the overlapping functional metrics (rag.py is authoritative).	Removes a full duplicate pass.
D	Fold bias into the consolidated judge or sample it (every Nth conv).	Removes 1 DeepEval call/conv.

Tier 2 — Waste removal (extra 15–25%)

#	Change	Effect
E	Reuse previewed personas in the run (cache by doc_sha+num_personas+role), or preview just 2–3 samples.	Halves s1 for preview→run users.
F	quality.py: one GEval/judge call scoring all criteria at once.	6 calls → 1 per conversation.
G	Cap DeepEval max_tokens to ~256; cap garak verdict to 5; tighten generator budgets.	Cuts output tokens on the biggest calls.
H	Make persona revision rarer; lower JSON retries from 3 → 2.	Trims s1 tail cost.

Tier 3 — Config / Infra (extra 30–50% of what remains)

#	Change	Effect
I	Honour the task tier: route fast/balanced to a small cheap model; reserve the judge model for scoring.	Often halves remaining cost.
J	Provider prompt caching for static blocks (system prompts, 133-pt taxonomy, RAG KB). Anthropic cache_control / Gemini implicit / OpenAI prefix caching.	Big discount on repeated prefixes.
K	Cheap pre-filters before the LLM (extend rag.py cosine short-circuit): skip judge when embedding similarity is decisively high/low; skip evals on refused/errored responses.	Removes whole calls.
L	Batch multiple conversations per judge call (the classifier already does this for failures).	Amortises fixed prompt overhead.
M	Sampling for expensive metrics; turn off litellm verbose/debug.	Linear token reduction.

6. Recommended Order of Execution

0. (DONE) Make the meter real — route DeepEval + the architecture-parser vision call through the LLMClient counter and log to MLflow. You cannot justify or reduce a cost you cannot see; this establishes the true baseline. Then: **A → C → B → I → E**. A+C alone typically halve the bill; I roughly halves what remains.

7. Expected Combined Savings

Action set	Token reduction (cumulative)
Tier 1 (A–D)	~60–80% of evaluation tokens
+ Tier 2 (E–H)	~70–85% total
+ Tier 3 (I–M)	~85–92% total vs. today

Numbers are directional; confirm against the now-accurate MLflow per-stage breakdown after one full run.

Generated for the METRON backend (metron-unified). Companion change: real MLflow LLM Ops tracking (see chat change log). RCA confirmed to make zero LLM calls; RAGAS confirmed dead code (no execution path).